Analysis of Heatpumps

Submitted to the
Albert-Ludwigs-Universität Freiburg

by Leander Marius Bürkin

universität freiburg

Bachelor's Thesis

Submitted on **April 29, 2026**
to the **Albert-Ludwigs-Universität Freiburg**
by **Paulo Rosas** (4141122)
Examined by **Prof. Dr. Mathias Fleury**

Systems Control and Optimization Laboratory
Department of Microsystems Engineering (IMTEK)
Faculty of Engineering
Albert-Ludwigs-Universität Freiburg

1. Enter your data.

2. Add as many bigskips as needed.

Enter your data int the Declaration.

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used.

Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg, April 29, 2026

Place, date	Leander Marius Bürkin

Acknowledgements

Thanks to my supervisor Max Mustermensch.

Further thanks go to the corporate design team at the Albert-Ludwigs-Universität Freiburg and Leander Marius Bürkin for this LaTeX-template.

Abstract

Zusammenfassung

Contents

1 Einleitung **1**
1.1 Warum SAT wichtig ist . . . 2
1.2 Motivation . . . 3

2 Preliminaries **5**
2.1 Conjunctive Normal Form (CNF) . . . 5
2.2 Kissat . . . 6
2.2.1 Kissat Architektur . . . 6
2.2.2 Kissat-Konfigurationsvarianten . . . 9
2.2.3 Messung der Solver-Leistung . . . 9
2.3 Satzilla . . . 9
2.3.1 Feature Extraction . . . 11
2.3.2 Machine Learning Model . . . 11
2.3.3 Training und Prediction Pipeline . . . 12

3 Satzilla Outcome **13**
3.1 Validierung mit Fake Solvern . . . 13
3.1.1 Design der fake Solver . . . 13
3.1.2 Resultate . . . 13
3.1.3 Interpretation . . . 15
3.2 Trainieren auf echten Kissat Solver Daten . . . 15
3.2.1 Dataset . . . 16
3.2.2 Training Konfiguration . . . 16
3.2.3 Testgenauigkeit . . . 16
3.3 Top-K-Genauigkeit: Der Wert des „fast Richtigen“ . . . 17
3.3.1 Resultate . . . 17
3.3.2 Interpretation . . . 19
3.4 Feature Importance Analysis . . . 20
3.4.1 Wichtigste Features . . . 20
3.4.2 Was die Features tatsächlich messen . . . 21
3.4.3 Feature Kategorien – Zusammenfassung . . . 25
3.5 Per-Solver Performance . . . 26
3.5.1 Interpretation . . . 27
3.6 Auswirkungen der Einbindung von LP-Features . . . 28
3.6.1 Ergebnis . . . 28
3.6.2 Interpretation . . . 28
3.7 Zusammenfassung der wichtigsten Ergebnisse . . . 29
3.8 Fazit . . . 30

4 Results **31**

5 Discussion **32**

6 Conclusion **33**

6.1 Tables and References 34
6.2 Auto-complete in VSCode 34
6.3 ", Quotations and Code 35
6.4 Clipboard Manager, Git and more 36
6.5 Glossary 36

Symbols **I**

List of Figures **II**

List of Tables **III**

List of Algorithms **IV**

Glossary **V**

References **VI**

Appendix **VII**

1 Einleitung

Das Erfüllbarkeitsproblem der Aussagenlogik (SAT, von englisch satisfabiality "Erfüllbarkeit") ist ein fundamentales Problem der Informatik mit kritischen Anwendungen in Software und Hardware Überprüfung.

1.1 Warum SAT wichtig ist

Trotz seiner theoretischen Bedeutung als erstes NP-vollständiges Problem (Cook, 1971) hat SAT breite praktische Anwendung in den Bereichen Hardware-Verifikation, Software-Testing, Planung und künstliche Intelligenz gefunden.[3]

Moderne SAT-solver haben in den letzten zwei Jahrzenten bemerkenswerte fortschritte gemacht; Löser wie Kissat (Biere et al., 2020) sind in der Lage, industrielle Instanzen mit Millionen von Variablen zu lösen.[1]

SAT hat breite praktische Anwendung gefunden, da sich viele Probleme aus der realen Welt als SAT-Instanzen formulieren lassen:

- **Hardware-Verifikation**: Nachweis, dass sich digitale Schaltungen korrekt verhalten
- **Softwaretests**: Erstellung von Testfällen, die bestimmte Pfade abdecken
- **Planung und Terminierung**: Ermittlung von Handlungsabläufen, mit denen Ziele erreicht werden
- **Kryptoanalyse**: Knacken kryptografischer Systeme durch das Lösen logischer Beschränkungen
- **Bioinformatik**: Analyse genetischer Sequenzen und Proteinfaltung

Die praktische Bedeutung von SAT hat jahrzehntelange Forschungen zur Entwicklung effizienter SAT-Löser vorangetrieben.

Der praktische Erfolg der SAT-Lösung ist weitgehend auf die bemerkenswerte Leistungsfähigkeit von CDCL-Lösern (Conflict-Driven Clause Learning) zurückzuführen. In der SAT-Forschung ist jedoch bekannt, dass kein einzelner Löser bei allen Instanzen allen anderen überlegen ist. Verschiedene Solver weisen komplementäre Leistungsmerkmale auf – ein Solver, der bei einer Instanzfamilie extrem schnell ist, kann bei einer anderen schlecht abschneiden. Diese Beobachtung hat zur Entwicklung von Algorithmusauswahlsystemen geführt, die automatisch den für eine bestimmte Instanz am besten geeigneten Solver auswählen.

Diese Erkenntnis hat zur Entwicklung des portfoliabasierten SAT-Lösens geführt, bei dem eine Sammlung von Solvern gepflegt wird und ein Auswahlmechanismus entscheidet, welcher Solver für eine bestimmte Instanz eingesetzt wird. Die zugrunde liegende Idee ist einfach, aber wirkungsvoll: Wenn wir vorhersagen können, welcher Solver bei einer Instanz die beste Leistung erbringt, können wir erhebliche Rechenzeit einsparen,

indem wir Solver mit schlechter Leistung vermeiden.

1.2 Motivation

In meinem ürsprunglichen Bachelorprojekt, was mit dieser Bachelorarbeit verknüpft ist, bestannt meine Aufgabe darin Satzilla im jahr 2025 zu wiederholen mit neuen Daten um dann zu vergleichen wie diese neuen vorhersagen mit den Alten aus dem Jahr 2024 übereinstimmen würden.

Dafür bekam ich von meinem Tutor Dr. Mathias Fleury den code von Satilla des Github repos und 400 cnf Dateien aus der enstprechenden webseite.

Die Arbeit war nicht immer einfach und ich hatte mit technischen Problemen zu kämpfen, so fehlten gewisse Dateien für die ich an dem Autor der Universität Aachen von Satzilla anschreiben musste. Des weiteren kam mein eigener PC an seine grenzen da er für die benötigten Vorhersagen nicht genug RAM hatte.

Dank der hilfe meiner beiden Tutoren konnte und der Bereitstellung eines PCs meiner Universität mit dem ich mich mit einer SSH Verbindung anmelden und benutzen konnte, konnte ich mein Arbeit zuende vollstellen.

Auch wenn das ergebniss meiner Arbeit nicht komplett zufriedenstellend war fand ich die Arbeit und Forschung interresant und herausfordernt.

Besonders interresiert hat mich, welche die Predictions waren die mir Satzilla jeweils für meine benchmarks empfohlen hat mit der dazugehörigen Laufzeit. Diese Daten konnte ich dann auswerten.

Für meine Bachelorarbeit, Predicting Kissat Performaance with Satzilla, wurde mir dann die volgenden zwei themen zugeweiesen:

Erstes habe ich getestet ob man Satzilla komplett auf gewissen Kissat Konfigurationen trainiren kann, sprich ob sie korrekte predicitons machen. Dafür musste ich als erstes auch testen ob Satzilla überhaupt mit seinem Machine Learning funktioniert da die ergebnisse im meinem Bachelorprojekt zweifel drauf ankommen ließen.

Zweitens habe ich getesten zu welchen ergebnissen man kommt wenn man Satzilla mit kissat trainiert und dann auf preprocessed daten anwendet, einaml die benchmarks

auf pre10k vereinfacht und dann auf pre100k vereinfacht, was bis dato niemand zuvor gemacht hat.

2 Preliminaries

In diesem Kapitel beschreibe ich die Grundlagen die für die Späteren Kapitel von bedeutung sein werden. Zuerst was CNF Dateien sind un wie sie aufgebaut sind, dann was Kissat und Satzilla sind und wie sie mit Dateien trainiert werden um Predicitons von Solvern zu machen.

2.1 Conjunctive Normal Form (CNF)

Bei einer Boolschen Formel, die aus durch UND- und ODER-Operatoren verbunden Variablen besteht, (AND $\wedge$, OR $\vee$, NOT $\neg$) geht es beim SAT-Problem um die Frage, ob es eine Zuorndung von Warheitswerten zu den Variablen gibt, bei der die gesamte Formel als Wahr ausgewertet wird.

Moderne SAT-Löser arbeiten in der Regel mit Formeln in konjunktiver Normalform (CNF). Eine CNF-Formel besteht aus:

- **Variablen**: Boolesche Variablen (z. B. x_1, x_2, ..., x_n)
- **Literale**: Eine Variable oder deren Negation (z. B. x_1 oder $\neg x_1$)
- **Klauseln**: Eine Disjunktion (ODER) von Literalen (z. B. $(x_1 \vee \neg x_2 \vee x_3)$)
- **Formel**: Eine Konjunktion (UND) von Satzteilen (z. B. $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$

Das CNF-Format ist sowohl ausdrucksstark (jede Boolesche Formel lässt sich in CNF umwandeln) als auch für algorithmisches Denken gut geeignet. Das folgende Beispiel zeigt eine einfache CNF-Datei:

```
p cnf 3 4
1 -2 0
-1 3 0
2 -3 0
-1 -2 -3 0
```

Dies stellt eine Formel mit 3 Variablen und 4 Klauseln dar. Die Zeile „p cnf“ gibt den Problemtyp und die Dimensionen an (3 Variablen, 4 Klauseln), und jede nachfolgende Zeile stellt eine Klausel dar, die mit 0 endet.

2.2 Kissat

Kissat ist ein hochmoderner SAT-Löser nach dem Prinzip des Conflict-Driven Clause Learning (CDCL), der von Armin Biere und seiner Forschungsgruppe an der Johannes Kepler Universität Linz entwickelt wurde. Der Kissat-SAT-Solver ist eine komprimierte und verbesserte Neuimplementierung von CaDiCaL in C. Er verfügt über verbesserte Datenstrukturen, eine optimierte Ablaufplanung der Datenverarbeitung sowie optimierte Algorithmen und Implementierungen.

Kissat belegte den ersten Platz in der Hauptkategorie des SAT-Wettbewerbs 2020 und den ersten Platz bei den unlösbaren Instanzen.

2.2.1 Kissat Architektur

Kissat implementiert den CDCL-Algorithmus mit mehreren ausgefeilten Optimierungen:

Kissat ist ein „einfacher und übersichtlicher Bare-Metal-SAT-Löser“, der in C geschrieben ist. Es handelt sich um eine Portierung von CaDiCaL zurück nach C mit verbesserten Datenstrukturen, einer optimierten Ablaufplanung der Datenverarbeitung sowie optimierten Algorithmen und Implementierungen.

1. Variblen Elimination

Die Variableneliminierung ist eine Vorverarbeitungstechnik, die eine SAT-Formel vereinfacht, indem eine Variable und alle Klauseln, die diese enthalten, entfernt werden, während neue Klauseln hinzugefügt werden, die die logische Äquivalenz bewahren.

Die Eliminierung von x ersetzt alle Klauseln, die x enthalten, durch die **Resolventen** zwischen jeder positiven und jeder negativen Klausel:

$$(x \vee A) \wedge (\neg x \vee B) \implies (A \vee B)$$

Die Variable x wird dann aus der Formel entfernt und die neuen Resolventen-Klauseln werden hinzugefügt.

Das ist in dieser Arbeit die wichtigste eigenschaft da weniger Variablen bedeutet eine geringerere Verzweigungsfaktor im CDCL-Suchbaum und durch diese simplifikation

kann die Lösung des Problems verschnellert werden.

Das Vorhandensein oder Fehlen der Variableneliminierung ist einer der Hauptunterschiede zwischen den Kissat-Konfigurationen, weshalb das Feature „reducedVars“ in unserem Satzilla-Modell als äußerst wichtiges Merkmal auftritt (Figure 7).

2. Subsumption Elimination

Durch Subsumption werden Klauseln entfernt, die sich logisch aus anderen, kürzeren Klauseln ergeben.

Beispiel: Die Klausel $(x \vee y)$ subsumiert $(x \vee y \vee z)$, da letztere immer dann wahr ist, wenn erstere wahr ist.

3. Probing (Failed Literal Propagation)

Untersuchung von Versuchen, Literale zuzuweisen und erzwungene Zuweisungen oder Unerfüllbarkeit zu erkennen.

4. Caching (Learned Clause Management)

Kissat speichert Klauseln und verwendet sie nach einem Neustart erneut, um doppelte Arbeit zu vermeiden.

5. Multi-Level Decision Heuristic

Die Entscheidungsheuristik legt fest, auf welche Variable und welche Phase als Nächstes verzweigt wird.

6. Restart Strategy

Bei einem Neustart wird der aktuelle Suchzweig verlassen und der Entscheidungsprozess neu gestartet, wobei die erlernten Klauseln beibehalten werden.

7. Inprocessing (Intermediate Simplification)

Inprocessing wendet Vereinfachungstechniken während der gesamten Lösung wiederholt an, nicht nur zu Beginn.

2.2.2 Kissat-Konfigurationsvarianten

Diese sechs Kissat-Varianten decken die wichtigsten Dimensionen des Konfigurationsraums ab – von minimaler Vorverarbeitung (noeliminate) über Standardverhalten (default) bis hin zu maximaler Stabilisierung (stable, stable2) und alternativen Heuristiken (target2) – und ermöglichen es Satzilla somit, für jede Instanz die passendste Strategie zu erlernen.

Dies ist der Grund weshalb genau diese sechs Kissat Solver für diese Arbeit ausgewählt, da sie am relevantesten sind.

Kissat bietet zahlreiche Konfigurationsoptionen, mit denen sich sein Verhalten anpassen lässt. Die in dieser Arbeit verwendeten Varianten stellen unterschiedliche Kompromisse zwischen dem Aufwand für die Vorverarbeitung und der Rechengeschwindigkeit dar:

Variante	**Haupteigenschaften**
kissat_default	Standardkonfiguration mit ausgewogenen Einstellungen
kissat_noeliminate	Deaktiviert die Variableneliminierung
kissat_nostable	Deaktiviert den stabilen Modus
kissat_stable	Aktiviert den stabilen Modus für eine Suche
kissat_stable2	Erweiterter Stabilitätsmodus mit zusätzlichen Heuristiken
kissat_target2	Optimiert für eine bestimmte Problemklasse

Table 1: Kissat-Konfigurationsvarianten für diese Arbeit

2.2.3 Messung der Solver-Leistung

Die Leistung eines Solvers wird in der Regel anhand der Gesamtlaufzeit oder der CPU-Zeit gemessen. In dieser Arbeit verwende ich die vom Feld „process-time“ von Kissat gemeldete Laufzeit, die die zur Lösung der Instanz aufgewendeten CPU-Sekunden angibt. Zeitüberschreitungen (z. B. 3600 Sekunden, 7200 Sekunden) deuten darauf hin, dass der Solver die Berechnung nicht innerhalb der zugewiesenen Zeit abgeschlossen hat. Daraus schließt man das das Problem unsatisfiable ist, sprich der solver konnte dafür keine Lösung finden die das Problem erfüllen könnte.

2.3 Satzilla

Satzilla (Xu et al., 2008) ist ein wegweisendes System, das maschinelles Lernen auf das Problem der Solverauswahl anwendet. Die zentrale Erkenntnis besteht darin, dass

SAT-Instanzen durch Merkmale charakterisiert werden können, die die Leistung des Solvers vorhersagen. Durch das Lernen aus historischen Laufzeitdaten kann Satzilla den besten Solver für bisher unbekannte Instanzen auswählen.[4]

2.3.1 Feature Extraction

Satzilla extrahiert aus jeder CNF-Formel über 100 Merkmale, die in mehrere Kategorien unterteilt sind:

Kategorie	Features	Was sie messen
Basic	Anzahl der Variablen, Anzahl der Klauseln, Verhältnis	Größe und Dichte
Variable-Clause Graph (VCG)	Mittelwert, Entropie, Min/-Max	Verbindungsstruktur eines Graphen
Clause Graph (CG)	Klauselverbindungen, Cluster	Wechselwirkungen zwischen Klauseln
Variable Graph (VG)	Variable co-occurrence	Variable Beziehungen
Horn Features	Horn-Klausel-Bruch, Eigenschaften der Horn-Variablen	Besondere Struktur
Probing Features	Ausbreitungstiefe der Einheit, reduzierte Variablen	Eigenschaften des Suchraums
Local Search	Leistungskennzahlen von GSAT/SAPS	Die Landschaft der lokalen Suche
LP Features	Relaxationswerte der linearen Programmierung	Kontinuierliche Entspannung

Table 2: SAT Instance Features Categories

Diese Merkmale werden von einem speziellen Merkmalsextraktor (Merkmalsexekutivdatei) berechnet, der die CNF-Datei analysiert und eine CSV-Datei ausgibt, die eine Zeile pro Instanz enthält.

2.3.2 Machine Learning Model

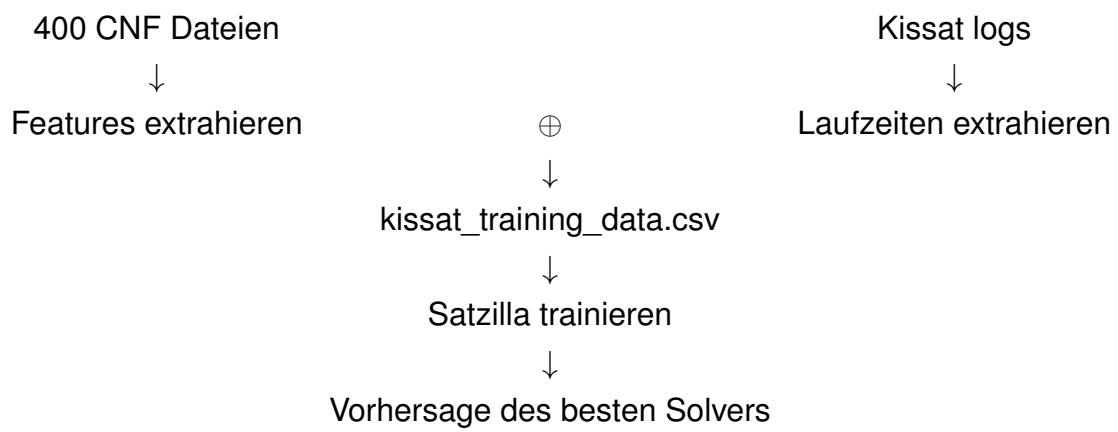
Satzilla nutzt „Random Forest“ (Breiman, 2001) als Lernalgorithmus. „Random Forest“ ist ein Ensemble-Verfahren, bei dem mehrere Entscheidungsbäume erstellt und deren Vorhersagen zusammengefasst werden. Wesentliche Vorteile für diese Anwendung:

- Unterstützt gemischte Merkmalstypen (kontinuierlich, kategorial)
- Unempfindlich gegenüber irrelevanten Merkmalen (der Algorithmus ignoriert verrauschte Eingaben)
- Zeigt die Merkmalsbedeutung an (welche Merkmale am wichtigsten sind)
- Verhindert Überanpassung durch geeignete Hyperparameter

Im ursprünglichen Satzilla sagt das Modell die Laufzeit direkt voraus. Für die Solverauswahl müssen wir jedoch nur wissen, welcher Solver am schnellsten ist – nicht die genaue Laufzeit. Diese Arbeit konzentriert sich auf die Genauigkeit der Solverauswahl.

2.3.3 Training und Prediction Pipeline

Die gesamte Satzilla-Pipeline besteht aus:



Mithilfe des SAT-Features-Extraktors habe ich 139 Features aus 400 CNF-Benchmarks extrahiert. Unabhängig davon habe ich die Protokolle des Kissat-Lösers ausgewertet, um die genauen Laufzeiten für sechs Lösungsvarianten bei denselben Instanzen zu ermitteln.

Nachdem ich die Merkmals- und Laufzeitdaten über eine Namenszuordnung abgeglichen hatten, identifizierte ich dann den schnellsten Solver pro Instanz und trainierten einen Random-Forest-Klassifikator. Daraus entstand ein Satzilla-Modell, das anhand der Merkmale einer neuen Instanz vorhersagt, welcher Kissat-Solver die beste Leistung erbringen wird.

3 Satzilla Outcome

In diesem Kapitel beschreibe ich wie ich Satzilla mit sogenanten Fake Solvern valediert habe und anschließend wie ich Satzilla mit kissat trainiert habe.

Zum schluss beschreibe ich die Features und ihre Wichtigkeit die ich durch das Training von Satzilla bekommen habe.

3.1 Validierung mit Fake Solvern

Da ich in meinem ursprünglichen Bachelorprojekt Predictions in Satzilla hatte die nicht mit den tatsächlichen vorhersagen übereinstimmten (so wurde fälschlicherweiße ISAsat alle der beste Solver für viele Probleme empfohlen) riet mir mein Tutor Dr. Mathias Fleury zu testen ob Satzilla überhaupt richtig funktioniert und passende vorhersagen machen kann.

Bevor ich Satzilla auf die Laufzeitdaten realer Solver angewandt haben, habe ich zunächst die gesamte Pipeline anhand künstlich erzeugter Solver-Verhaltensmuster validiert. Dieser Schritt war entscheidend, um sicherzustellen, dass die Merkmalsextraktion, die Datenausrichtung und der Lernmechanismus korrekt funktionieren.

3.1.1 Design der fake Solver

Ich habe künstliche Solver erstellt, die explizite, bekannte Beziehungen zu den Eigenschaften der Instanzen aufweisen (z. B. zur Anzahl der Variablen). Auf diese Weise konnte ich eine Referenzgröße definieren: Für jede Instanz wussten ich genau, welcher Solver als der schnellste anzusehen war.

- **3-Solver-Test**: Auf kleine, mittlere und große Instanzen spezialisierte Solver, basierend auf Schwellenwerten für die Variablenanzahl (33. und 67. Perzentil).
- **4-Solver-Test**: Zwei Solver basieren auf der Variablenanzahl und zwei auf der Klauselanzahl, wobei eine klare Hierarchie besteht (Variablenspezialisten dominieren).

3.1.2 Resultate

Genauigkeit der Ergebnisse der Fake Solver:

Eine Genauigkeit von 99 % bedeutet, dass Satzilla in 99 von 100 Fällen den schnellsten Fake Solver korrekt identifiziert, was beweist, dass die Merkmalsextraktion und das

Test	Solver Count	Accuracy
Sprungfunktion mit beliebigen Schwellenwerten (100k/200k)	3	65%
Lineare Funktion (laufzeit in relation zur Anzahl der Variablen)	3	99%
Sprungfunktion mit datenabhängigen Schwellenwerten	3	99.25%
Schrittweise Hierarchie (Variablen + Klauseln)	4	99.75%

Table 3: Testergebnisse für verschiedene Auswahlstrategien

Training einwandfrei funktionieren.

Wichtigste Features der Fake Solver:

Rang	Feature	Relevanz
1	nvarsOrig	13.07%
2	VCG-CLAUSE-min	8.46%
3	nvars	7.79%

Table 4: Testergebnisse der wichtigsten Features

Je höher der Prozentsatz des Features ist, desto stärker stützt sich das Modell bei seinen Vorhersagen auf dieses Feature.

3.1.3 Interpretation

Die hohe Genauigkeit ($\geq 99\%$) in drei von vier Tests bestätigte, dass:

- Die Feature extraction erfasst die Eigenschaften der Instanzen korrekt. nvarsOrig führt die direkte Messung der Anzahl der ursprünglichen Variablen durch – das wichtigste Unterscheidungsmerkmal, nvars die Anzahl der Variablen nach dem Preprocesing und VCG-CLAUSE-min den minimaler Klauselgrad in einem Graphen mit variablen Klauseln (korreliert mit der Größe)
- Die Trainingspipeline (Random-Forest-Klassifikator, Kreuzvalidierung) funktioniert wie vorgesehen.
- Das Scheitern des ersten Tests (65 %) war auf schlecht gewählte Schwellenwerte (100k/200k) zurückzuführen, die zu einem unausgewogenen Datensatz führten – nicht auf einen Fehler in Satzilla selbst.

Fazit: Die Satzilla-Pipeline funktioniert einwandfrei und ist bereit für das Training mit echten Solver-Daten.

3.2 Trainieren auf echten Kissat Solver Daten

Nach der Validierung habe ich Satzilla mit sechs Konfigurationen des Kissat-Solvers trainiert:

- kissat_default
- kissat_noeliminate
- kissat_nostable
- kissat_stable
- kissat_stable2

- kissat_target2

3.2.1 Dataset

- **397 Instanzen** mit vollständigen Features, die Features wurden mithilfen des Satzilla tools extrahiert
- **Laufzeiten** für jeden Solver bei jeder Instanz (aus tatsächlichen Ausführungen)
- **Train/test split**: 80 % Training (317 Fälle), 20 % Test (80 Fälle), stratifiziert, um die Verteilung der Solver beizubehalten

3.2.2 Training Konfiguration

- **Model**: Random Forest Classifier (200 trees, max depth 20)
- **Features**: 131 strukturelle Features (ohne LP-Merkmale, gemäß rt_pred.py)
- **Ziel**: Für jede Instanz der Solver mit der kürzesten Laufzeit (der schnellste)

3.2.3 Testgenauigkeit

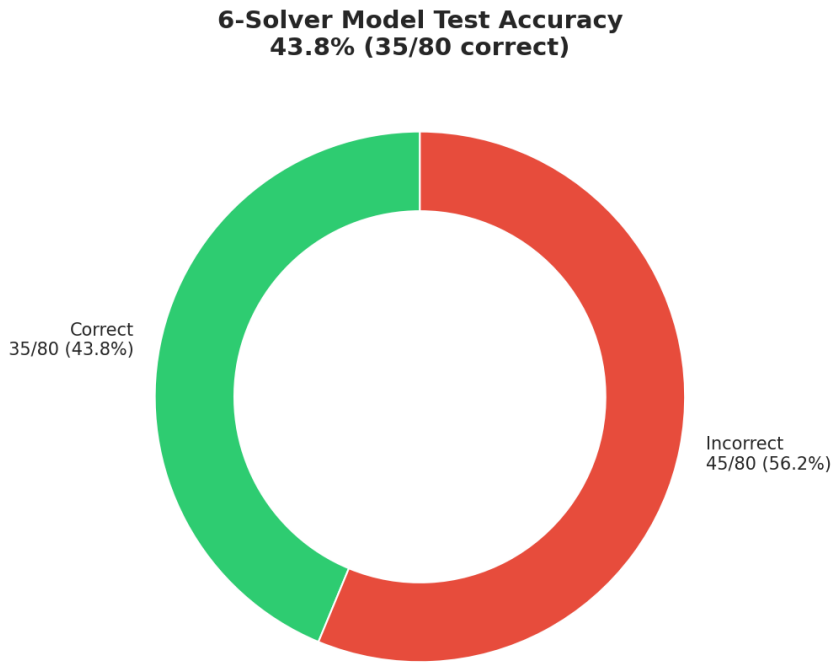


Figure 1: Testgenauigkeit der verschiedenen Strategien

Was wir in dieser Graphik sehen ist das in 43.8% der Fälle Satzilla trainier auf Kissat tatstächlich den schnellsten und somit besten Solver empfiehlt.

Dies zeigt uns das selbst bei begrenzten Trainingsdaten (397 Instanzen für 6 Solver)

schneidet Satzilla deutlich besser ab als eine zufällige Auswahl.

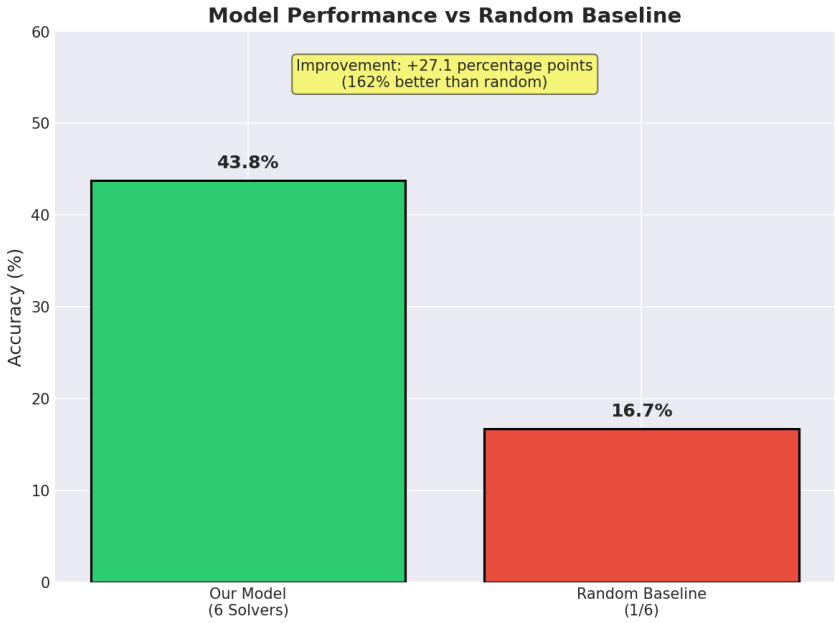


Figure 2: Vergleich mit Random Selection

Somit ist die Auswahl von Satzilla für den schnellsten solver um 162% besser als einen der sechs Solver durch zufallsprinzip auszusuchen.

Satzilla trainiert auf Kissat zeigt das es zuverlässig den schnellsten solver empfehlen kann.

3.3 Top-K-Genauigkeit: Der Wert des „fast Richtigen“

Bei der Lösung von SAT-Problemen in der Praxis reicht es oft aus, wenn das Ergebnis nahe am optimalen Lösungsvorschlag liegt. Ein Nutzer kann die ersten k Empfehlungen nacheinander ausprobieren. Daher habe ich auch die Genauigkeit der Top-2- und Top-3-Ergebnisse gemessen – also die Wahrscheinlichkeit, dass der tatsächlich schnellste Lösungsvorschlag unter den ersten k Vorhersagen auftaucht.

3.3.1 Resultate

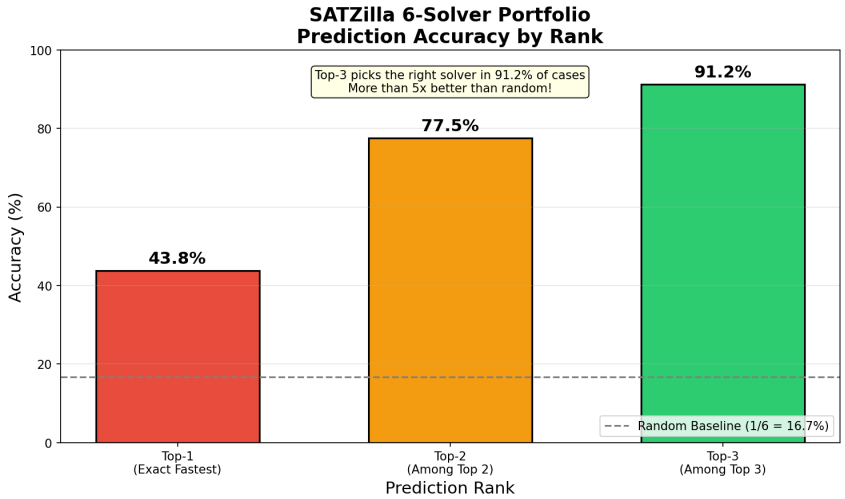


Figure 3: Vergleich der Top 3 besten Solver

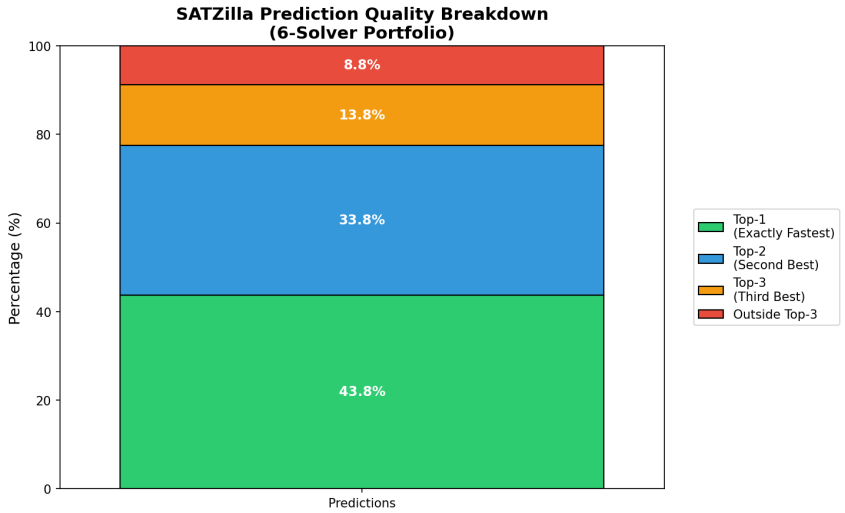


Figure 4: Zusammenfassung der Besten Solver

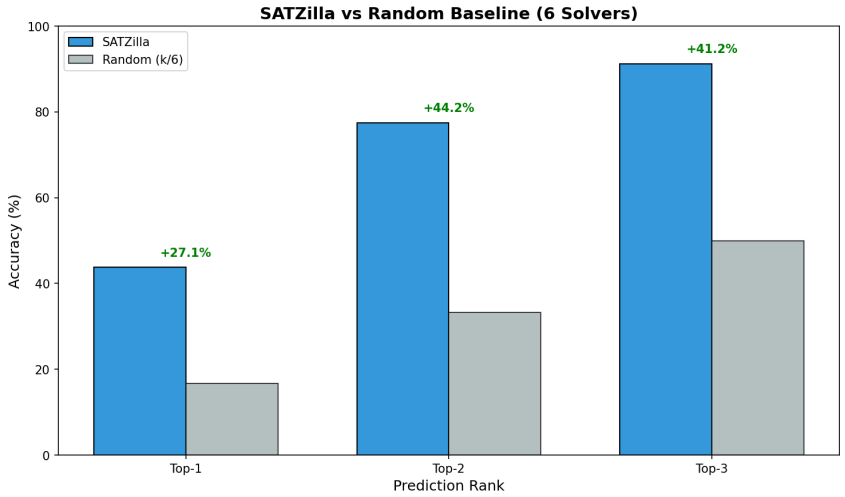


Figure 5: Vergleich der drei besten Solver mit Zufallsprinzip

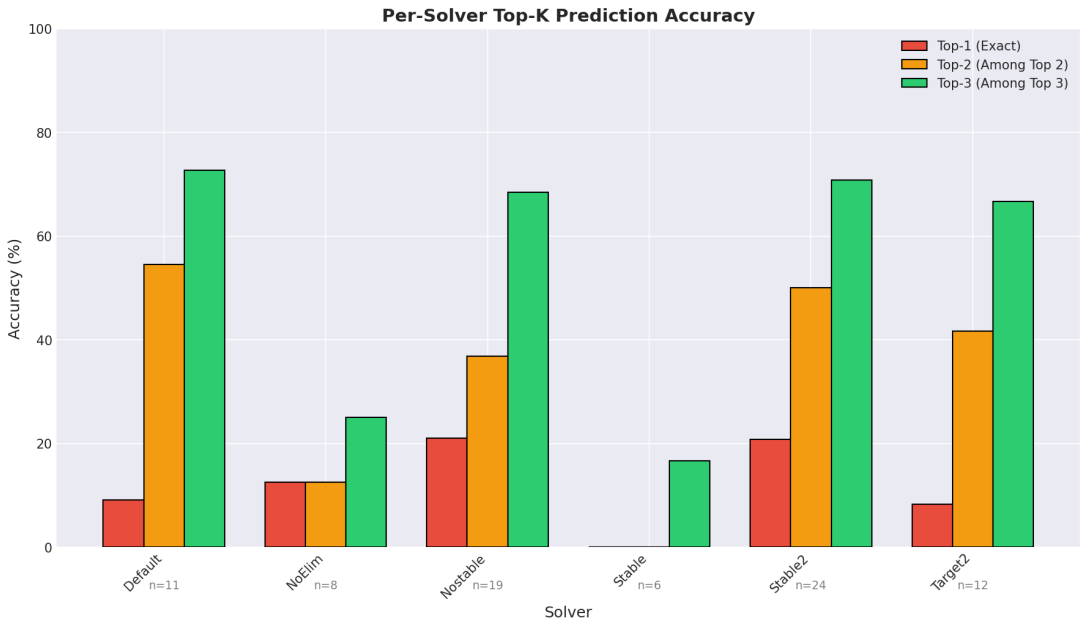


Figure 6: Für jeden solver Top 3 Vorhersagen

3.3.2 Interpretation

- In **85% der Testfälle**, befindet sich der Schnellste vorgeschlagene Solver von Satzilla in den **Top drei Vorhersagen**
- Das bedeutet, dass ein Nutzer, der die drei am häufigsten empfohlenen Solver ausprobiert, in den allermeisten Fällen eine nahezu optimale Leistung erzielen wird
- Selbst wenn Satzilla nicht den absolut schnellsten Solver vorhersagt, **empfiehlt es so gut wie nie den schlechtesten Solver**

Fazit: Satzilla ist nicht nur besser als Zufallsverfahren, sondern auch praktisch nützlich für die portfoliogestützte SAT-Lösung.

3.4 Feature Importance Analysis

Für die Interpretierbarkeit ist es wichtig zu verstehen, welche Features die Auswahl des Solvers beeinflussen. Ich haben die Merkmalsbedeutungswerte des trainierten Random-Forest-Modells analysiert.

3.4.1 Wichtigste Features

Die analyse der Features zeigt auf, welche Eigenschaften einer Instanz die Auswahlentscheidungen des Satzilla-Lösers am stärksten beeinflussen. Dies ist nicht nur für die Interpretierbarkeit des Modells von Bedeutung, sondern auch für das Verständnis, was eine SAT-Instanz für verschiedene Kissat-Konfigurationen „einfach“ oder „schwierig“ macht.

In unserem trainierten Random-Forest-Modell gibt die Feature wichtigkeit an, wie häufig ein Feature bei der Entscheidung über die Aufteilung in allen Bäumen herangezogen wird. Eine höhere Bedeutung bedeutet, dass das Merkmal stark damit korreliert, welcher Solver am schnellsten ist.

In der folgenden Tabelle sind die 15 wichtigsten Features nach ihrem Beitrag zu den Vorhersagen des Modells aufgelistet:

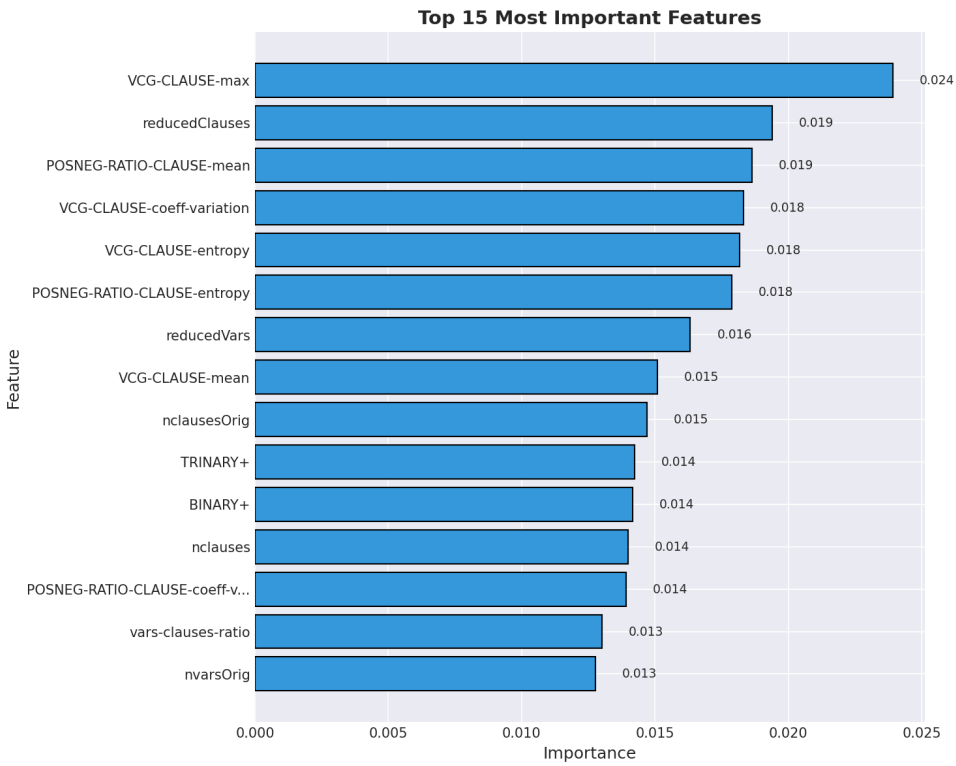


Figure 7: Wichtigste Features für die Vorhersagen

3.4.2 Was die Features tatsächlich messen

Um zu verstehen, warum diese Funktionen wichtig sind, fassen wir sie nach Kategorien zusammen:

Graph-Based Features (VCG – Variable-Clause Graph)

Der **Variable-Clause-Graph (VCG)** stellt die SAT-Instanz als zweigeteilten Graphen dar, in dem Variablen und Klauseln Knoten sind und Kanten das Vorkommen von Variablen in Klauseln anzeigen. Aus diesem Graphen abgeleitete Merkmale erfassen die strukturelle Komplexität.

Feature	**Was es berechnet**	**Warum es für die Auswahl des Solvers wichtig ist**
VCG-CLAUSE-max	Maximaler Klauselgrad (Anzahl der Variablen, die eine Klausel enthält)	Hohe Werte weisen auf weitreichende Klauseln hin, die sich auf die Ausbreitung und Auflösung auswirken
VCG-CLAUSE-coeff-variation	Abweichungen im Grad der Klauseln	Eine ungleichmäßige Verteilung der Klauseln verändert die Art und Weise, wie CDCL-Löser den Suchraum durchlaufen
VCG-CLAUSE-entropy	Entropie der Verteilungshäufigkeit von Klauseln	Hohe Entropie $\rightarrow$ irreguläre Struktur $\rightarrow$ abweichendes Verhalten des Solvers
VCG-CLAUSE-mean	Durchschnittlicher Grad der Klausel	Größere Klauseln erschweren einigen Solvern tendenziell die Berechnung von Instanzen
VCG-CLAUSE-min	Mindestgrad der Klausel (Einheitsklauseln)	Einheitsklauseln vereinfachen die Instanz und begünstigen bestimmte Lösungsstrategien

Table 5: VCG Klausen Features and ihre wichtigkeit

Auswertung: Features, die sich auf Klausen-Graph beziehen, dominieren die Spitzenplätze (5 Features unter den Top 15). Dies deutet darauf hin, dass die Art und Weise, **wie sich Satzteile mit Variablen verknüpfen**, der aussagekräftigste Prädiktor dafür ist, welche Kissat-Variante die beste Leistung erbringt.

Preprocessing Features

Kissat wendet verschiedene Inprocesing an (Variableneliminierung, Subsumption usw.). Die Anzahl der nach der Vorverarbeitung verbleibenden Variablen und Klauseln hat einen starken Einfluss auf das Verhalten des Solvers.

Feature	Was es berechnet	Warum es für die Auswahl des Solvers wichtig ist
reducedVars	Nach der Vereinfachung verbleibende Variablen	Einige Varianten sind für reduzierte Formeln optimiert
reducedClauses	Nach der Vereinfachung verbleibende Klauseln	Gibt an, inwieweit eine Vereinfachung möglich war

Table 6: Preprocessing Features and ihre wichtigkeit

Auswertung: Das Vorhandensein von **reducedVars** auf Rang 2 bestätigt, dass der Grad der Vereinfachung der Instanz entscheidend ist. Solver wie kissat_noeliminate verzichten auf Eliminierungen, weshalb sie bei stark reduzierbaren Instanzen eine andere Leistung zeigen.

Polarity Features (POSNEG-RATIO-CLAUSE*)

Diese Features erfassen das Gleichgewicht zwischen positiven und negativen Literalen innerhalb von Sätzen.

Feature	Was es berechnet	Warum es für die Auswahl des Solvers wichtig ist
reducedVarPOSNEG-RATIO-CLAUSE-entropy	Entropie der Polaritätsverteilung	Ausgewogene vs. unausgewogene Polarität beeinflusst Entscheidungsheuristiken
POSNEG-RATIO-CLAUSE-coeff-variation	Varianz bei den Polaritätsverhältnissen	Einige Kissat-Varianten weisen polaritätsabhängige Verzweigungen auf
POSNEG-RATIO-CLAUSE-mean	Durchschnittliches Polaritätsverhältnis	Stark verzerrte Fälle können bestimmte Heuristiken begünstigen

Table 7: Preprocessing Features and ihre wichtigkeit

Auswertung: Die Polaritätsfeatures zählen zu den zehn wichtigsten Merkmalen. Sie erfassen das Verhältnis von positiven und negativen Literalen innerhalb von Klauseln, was sich auf die Verzweigungsheuristiken und die Leistung des Solvers auswirkt.

Clause Length Distribution (BINARY+, TRINARY+)

Diese Features zählen Klauseln mit einer bestimmten Länge.

Feature	Was es berechnet	Warum es für die Auswahl des Solvers wichtig ist
BINARY+	Anteil der binären Klauseln (2 Literale)	Binäre Klauseln ermöglichen eine effiziente Einheitsfortführung
TRINARY+	Anteil der ternären Bedingungen (3 Literale)	Längere Klauseln vergrößern den Suchraum und beeinflussen den Lernprozess

Table 8: Klausel länge Features and ihre wichtigkeit

Auswertung: Instanzen mit vielen binären Klauseln verhalten sich anders als solche, die überwiegend lange Klauseln enthalten. Einige Kissat-Varianten sind auf bestimmte Verteilungen der Klausellängen abgestimmt.

Basic Size Features (nvarsOrig, nclausesOrig)

Auch wenn sie nicht die wichtigsten Prädiktoren sind (wie in den Experimenten mit dem Fake-Solver), spielt die Größe der Rohinstanz dennoch eine Rolle.

Feature	**Was es berechnet**	**Warum es für die Auswahl des Solvers wichtig ist**
nvarsOrig	Ursprüngliche Anzahl der Variablen	Grobe Annäherung an die Gesamtkomplexität
nclausesOrig	Ursprüngliche Anzahl der Klauseln	Legt zusammen mit Variablen die Instanzdichte fest

Table 9: Basic size Features and ihre wichtigkeit

Auswertung: Basic Size Features erfassen die ursprüngliche Größe einer SAT-Instanz; sie bieten einen groben Anhaltspunkt für die Komplexität und die Eignung des Solvers und liefern einen bescheidenen, aber aussagekräftigen Beitrag zu den Entscheidungen des Modells.

3.4.3 Feature Kategorien – Zusammenfassung

Kategorie	Features in Top 15	Wichtigkeit (%)	Wichtige Erkenntnis
Graph (Clause)	5	∼8.2	Das Wichtigste: Die Verknüpfung von Klauseln bestimmt die Unterschiede zwischen den Solvern
Preprocessing	2	∼3.3	Entscheidend ist, inwieweit die Instanz die Sache vereinfacht
Polarity	3	∼4.9	Das Polaritätsgleichgewicht beeinflusst die Verzweigungsheuristik
Clause length	2	∼3.1	Der Unterschied zwischen binären und langen Klauseln ist entscheidend
Size	2	∼2.6	Still relevant, but not dominant
Local search	1	∼1.3	Liefert ein ergänzendes Härtesignal

Table 10: Feature Kategorien und ihre wichtigkeit

Zusammenfassend: Die Auswahl des Solvers hängt in erster Linie von den **Eigenschaften des Strukturgraphen ab**, insbesondere von denen, die sich auf die Klauseln beziehen. Vorverarbeitung, Polarität und die Verteilung der Klausellängen liefern wichtige sekundäre Hinweise. Die reine Instanzgröße ist zwar in vereinfachten Szenarien (Fake-Solver) nützlich, stellt bei echten Kissat-Solvern jedoch nur einen von vielen Faktoren dar.

3.5 Per-Solver Performance

Nicht alle Solver lassen sich gleichermaßen leicht vorhersagen. Die folgende Tabelle zeigt die Genauigkeit der einzelnen Solver im Testdatensatz:

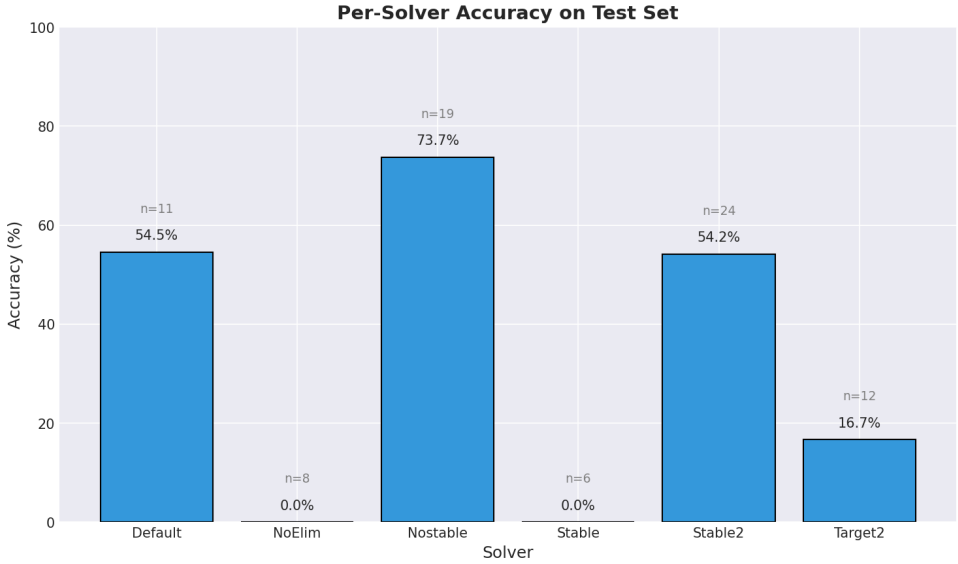


Figure 8: Vorhersagen Genauigkeit nach Solver

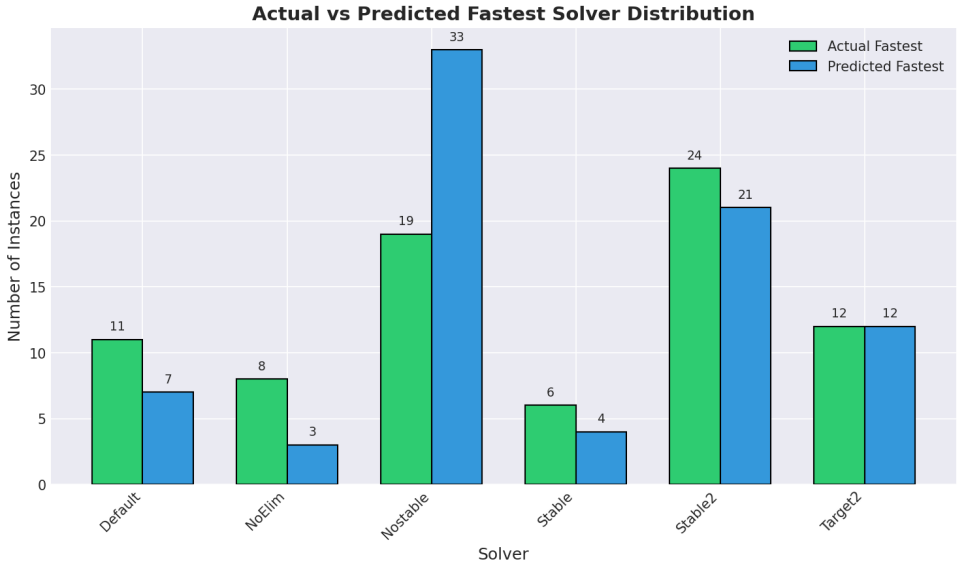


Figure 9: Vorhersagen Korrektheit nach Solver

3.5.1 Interpretation

- **Gut repräsentierte Solver** (mit mehr Trainingsinstanzen) erzielen eine höhere Genauigkeit
- kissat_nostable kommt in 96 Trainingsinstanzen vor $\rightarrow$ **74 % Testgenauigkeit**
- kissat_stable and kissat_noeliminate kommen nur in 28 and 39 training instances vor $\rightarrow$ **0% vorhersagen Genauigkeit**
- Dies macht eine **Einschränkung der Daten** deutlich: Bei insgesamt nur 397 Instanzen verfügen einige Solver über zu wenige Beispiele, als dass das Modell ihr Verhalten zuverlässig erlernen könnte

Folgerung: Eine Vergrößerung des Trainingsdatensatzes würde wahrscheinlich die Genauigkeit verbessern, insbesondere bei unterrepräsentierten Solvern.

3.6 Auswirkungen der Einbindung von LP-Features

Bei der Feature extraction der einzelnen Benchmarks ist mir aufgefallen das man Features extrahieren konnte die nicht in der Liste von features von rt_pred.py in Satzilla vorkommen aber denoch im Paper zu Satzilla.

LPFeatures (Features der linearen Programmierung) sind Instanzmerkmale, die sich aus der Lösung einer linearen Programmierungsrelaxation des SAT-Problems ableiten.

Mit den sieben LP-Features - LP_OBJ, LPSLack-mean, LPSLack-coeff-variation, LPSLack-min, LPSLack-max, lpIntRatio und lpTIME - hatte ich dann instesammt alle 138 Features und nicht nur 131 die in rt_pred aufgelistet sind.

Um zu testen ob die Fehlenden LPFeatures meine vorhersagen verbessern konnten habe ich mein auf Kissat trainiertes Satzilla wieder auf die gleichen Instanzen getestet (dieses mal mit 138 Features)

3.6.1 Ergebnis

Feature Set	Vorhersage Genauigkeit
131 features (ohne LP)	43.75%
138 features (+7 LP)	42.50%

Table 11: Unterschied der Vorhersage mit LP-Features

3.6.2 Interpretation

- Die LPFeatures haben die Genauigkeit **nicht verbessert**, sondern leicht verringert
- Die Features Relevanz der LP-Merkmale lag jeweils unter 0,35 %
- Die Extraktion von LP-Features ist rechenintensiv.

Fazit: Der Verzicht auf LP-Features (wie im ursprünglichen rt_pred.py) ist gerechtfertigt. Strukturelle Merkmale reichen für diese Aufgabe aus. Das könnte auch der Grund sein weshalb sie aus der liste von Features ausgeschlossen wurden.

3.7 Zusammenfassung der wichtigsten Ergebnisse

Die Satzilla-Pipeline funktioniert mit einer Genauigkeit von über 99% bei künstlichen Solvern (Fake Solvern).

Bei realen Kissat-Solvern erreicht Satzilla eine Top-1-Genauigkeit von 43,75%, was besser ist als das Zufallsprinzip. Besonders bemerkenswert ist die Top-3-Genauigkeit von 85%, die einen äußerst praktischen Nutzen für die SAT-Lösung bietet.

Als wichtigste Features für die Solver-Auswahl wurden strukturelle Merkmale (VCG, clause ratios) identifiziert. LP-Features erwiesen sich hingegen als nicht nutzbringend, da sie die Genauigkeit verringern und die Kosten erhöhen.

Eine Einschränkung der Datenlage besteht darin, dass für bestimmte Solver mehr Instanzen benötigt werden. Insgesamt sind die Top-3-Vorhersagen von Satzilla in 85% der Fälle erfolgreich.

3.8 Fazit

In diesem Kapitel wurde das Satzilla-Framework sowohl anhand künstlicher als auch anhand realer Solver-Daten evaluiert. Die wichtigsten Ergebnisse sind:

- **Pipeline-Validierung**: Validierung der Pipeline mit künstlichen Solvern (Fake Solver), die die Korrektheit bestätigt.
- **Training**: Erfolgreiches Training auf sechs echten Kissat-Solvern.
- **Top-3-Genauigkeit**: 85%, was den praktischen Nutzen für das portfoliobasierte SAT-Lösen demonstriert.
- **Feature-Wichtigkeit**: Identifikation struktureller Features als wichtigste Merkmale für die Solver-Auswahl.
- **LP-Features**: Empirische Rechtfertigung für den Ausschluss von LP-Features aus dem Feature-Set.

Zwar bleibt die genaue Vorhersage des schnellsten Solvers angesichts begrenzter Daten (44 % Top-1) eine Herausforderung, doch identifiziert Satzilla zuverlässig leistungsstarke Solver unter den Top-3-Empfehlungen. Dies macht es zu einem wertvollen Werkzeug für Nutzer, die es sich leisten können, eine kleine Auswahl an Solvern auszuprobieren, anstatt sich auf eine einzige Standardkonfiguration zu verlassen.

Abschließende Aussage: *Satzilla lernt erfolgreich vorherzusagen, welcher Kissat-Solver für eine gegebene SAT-Instanz voraussichtlich am schnellsten ist. Auch wenn die Vorhersage nicht exakt richtig ist, empfiehlt es konsistent hochwertige Solver und erreicht eine Erfolgsquote von 85%, wenn die drei besten Vorhersagen berücksichtigt werden.*

4 Results

5 Discussion

6 Conclusion

ToDos

Select the correct language in the first rows of `main.tex`. The % makes a line a comment.

Generate a PDF (Visual Studio Code: click on the green arrow in the top right corner).

Check the PDF (Visual Studio Code: click on the two rectangles with the magnifying glass in the top right corner).

Talk about this template with your supervisor: Should you add a declaration for the use of AI? Currently no official AI-declaration is given. You can find one in the pull requests of the github repository.

With CTRL pressed you can click on the PDF and jump to the corresponding place in your `.tex` files.

Check the Errors. (Visual Studio Code: bottom left) There should be one warning because the first ToDo pushes down the second one: `LaTeX Marginpar on page 2 moved.`

In the `main.tex` you can press ALT + SHIFT and the ARROW UP and DOWN keys to create a multiline cursor, comment out all the sections you are currently not working on and remove the multiline cursor with ESC.

Keep the following examples and todos till the end and check them out sometimes.

Adjust your License Notes to reflect the used programs.

For Printing: Remove the `todonotes`-package in `setup.tex` to find all ToDos.

Table 12: A nice table with a nice caption.

Value & Unit	Value	Unit

For Printing: Choose the correct formating (search `print` in `setup.tex`).

You can remove the glossary and the symbolslist to make it easier, especially in a Bachelor's Thesis.

6.1 Tables and References

Table 12

This is a cited book [2].

Store and display plots and other graphs as `.pdf` files.

Include Algorithm 1 in the same way you include equations like Equation 1:

$$\begin{aligned} y &= 3x^2 + 5x^2 + 4x + 4x + 7 + 1 \\ &= 8x^2 + 8x + 8 \\ y &= 8 \cdot (x^2 + x + 1) \end{aligned} \tag{1}$$

6.2 Auto-complete in VSCode

There are also some auto-complete features: type @ + letter to get the letters greek version.

Type @v + letter for the alternative form (like φ for ϕ).

You can also use shortcuts for environments like

$$BAL \tag{2}$$

Type BAL and select the correct option from the autocomplete. Maybe you have to try various ones. The B is the base command and AL is the abbreviation for align.

If you want an environment without an index, you can use BS as base command.

Same for changes to your font, like bold or italic: FIT gives *text*

6.3 ", Quotations and Code

Use " or `file-names`, `file_names` and `code_snippets`.

Algorithm 1: The Colors from the Corporate Design of the University of Freiburg. The keywords are bold and the comments blue. This can be changed in `setup.tex`.

```
from dataclasses import dataclass

# https://cd.uni-freiburg.de/farben/

@dataclass
class MainColors:
    blue = "#344A9A"
    black = "#000000"
    white = "#FFFFFF"

@dataclass
class BackgroundColors:
    blue_100_percent = "#344A9A"
    blue_80_percent = "#5D6BAD"
    blue_60_percent = "#868DC2"
    blue_80_percent = "#AFB1D8"
    blue_20_percent = "#D7D8EC"
    dark_blue = "#00004a"
    sand_100_percent = "#f6f1e3"
    sand_50_percent = "#faf8f1"

@dataclass
class AdditionalColors:
    green_100_percent = "#00a082"
    green_80_percent = "#27b29b"
    green_60_percent = "#7bc6b4"
```

```
green_80_percent = "#afdace"
green_20_percent = "#daede7"
brown_100_percent = "#8f6b30"
brown_80_percent = "#a58959"
brown_60_percent = "#bca683"
brown_80_percent = "#d2c4ac"
brown_20_percent = "#e9e1d6"
yellow_100_percent = "#ffe863"
yellow_80_percent = "#ffed82"
yellow_60_percent = "#fff1a1"
yellow_80_percent = "#fff6c1"
yellow_20_percent = "#fffae0"
pink_100_percent = "#f5c2ed"
pink_80_percent = "#f7cef1"
pink_60_percent = "#f9daf4"
pink_80_percent = "#fbe7f8"
pink_20_percent = "#fdf3fb"
```

6.4 Clipboard Manager, Git and more

Clipboard Manager help with copying multiple different things at once. On Windows you can press WINDOWS + V to activate the history and choose older things you copied.

Use Git (click here).

6.5 Glossary

Click them, the page numbers in the glossaries are also clickable.

name one

name one

name one

long form of name two (short2)

short2

short2

$a\ symbol$

$a\ symbol$

$a\ symbol$

Big Letter at start: Name one

plural of name one

name one

Big Letter at start: Short2

short2

short2s

Big Letter at start: $A\ symbol$

$a\ symbol$

Symbols

Notation	Description	Page List
$a\ symbol$	description symbol	37, 38

List of Figures

1 Testgenauigkeit der verschiedenen Strategien 16
2 Vergleich mit Random Selection 17
3 Vergleich der Top 3 besten Solver 18
4 Zusammenfassung der Besten Solver 18
5 Vergleich der drei besten Solver mit Zufallsprinzip 18
6 Für jeden solver Top 3 Vorhersagen 19
7 Wichtigste Features für die Vorhersagen 20
8 Vorhersagen Genauigkeit nach Solver 26
9 Vorhersagen Korrektheit nach Solver 26

List of Tables

1 Kissat-Konfigurationsvarianten für diese Arbeit 9
2 SAT Instance Features Categories 11
3 Testergebnisse für verschiedene Auswahlstrategien 14
4 Testergebnisse der wichtigsten Features 15
5 VCG Klausen Features and ihre wichtigkeit 21
6 Preprocessing Features and ihre wichtigkeit 22
7 Preprocessing Features and ihre wichtigkeit 23
8 Klausel länge Features and ihre wichtigkeit 23
9 Basic size Features and ihre wichtigkeit 24
10 Feature Kategorien und ihre wichtigkeit 25
11 Unterschied der Vorhersage mit LP-Features 28
12 A nice table with a nice caption. 34

List of Algorithms

1 The Colors from the Corporate Design of the University of Freiburg. The keywords are bold and the comments blue. This can be changed in `setup.tex`. 35

Glossary

Notation	Description	Page List
name one	description one	36, 38
short2	description two	36–38

References

[1] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. Cadical, kissat, paracooba, plingeling and treengeling entering the sat competition 2020. In *Proc. of SAT Competition 2020 - Solver and Benchmark Descriptions*, volume B-2020-1 of *Department of Computer Science Report Series B*, pages 50–53, Helsinki, 2020. University of Helsinki.

[2] Leander Bürkin. *LaTeX-Template*. Fachschaft TF Freiburg, 2025.

[3] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC '71)*, pages 151–158, New York, NY, USA, 1971. ACM.

[4] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Satzilla: Portfolio-based algorithm selection for sat. *Journal of Artificial Intelligence Research (JAIR)*, 32:565–606, 2008.

Appendix